

OBLIG 2

Oppgave 1.1

- Class

En klasse kan defineres som en plantegning i et javaprogram. Det er her vi definerer hvilke muligheter vi har når vi senere skal begynne å opprette objekter i koden. Slik som hvilke attributter et datasett tilhørende instansvariabler skal holde, samt hvilke egenskaper eller metoder som skal være tilgjengelige. Datasett tilhørende instansvariabler som blir opprettet i en klasse kan også defineres av andre klasser. Noe som gjør det mulig å knytte flere klasser sine egenskaper opp mot hverandre (f.eks. arving av klasser).

- Object (konseptet, ikke klassen)

Objekter er sin helhet en instans av en klasse. De er i stand til å ta i bruk alle tillatte egenskaper og attributter som vi har definert i den refererte klassen. Eksempelvis så vil en klasse som inneholder alle egenskaper til en ost gi oss muligheten til å opprette et objekt som spesifiserer hvilken ost det er snakk om, og videre hvilke verdier denne spesifikke osten holder på.

- Instansvariabel

Instansvariabler er variabler som vi deklarerer i en klasse. Dette vil ofte være hvilke attributter som et objekt kan/må inneholde ved opprettelse.

- Overloading

Overloading er når vi deklarerer flere metoder i en klasse med samme navn, men forskjellige antall parameter. Dette er praktisk med tanke på at det åpner for mer fleksible løsninger for å opprette objekter. Eksempelvis kan man ta for seg en klasse som heter Ost. I denne klassen har vi to metoder med samme navn som definerer vekt, men noen skriver vekten som int verdier og noen skriver det som string verdier. Vi kan åpne for begge muligheter ved å opprette to metoder som klarer hver sitt datasett, men som holder samme navn.

```
// I klassen
public int kgVekt(int x) {
    return x;
}

public String kgVekt(String x) {
    return x;
}

// I Main
public static void main(String[] args) {
    Ost ost = new Ost();

    int vektMedInt = ost.kgVekt(2);
    String vektMedString = ost.kgVekt("To");
}
```

- Overriding

Benyttes for å omdefinere en metode slik at egenskapen er tilpasset funksjonalitet for den inneværende klassen. Eksempelvis, ved å bruke Override funksjon kan vi få toString() metoden til å gi en beskrivelse av objektet som samsvarer med objektets inneværende klasse.

- Extends

Dette er et nøkkelord i Java som brukes for å deklareere arv. Med dette menes at vi kan gi en barneklasse de samme egenskapene som er deklart i en foreldreklasse ved å bruke extends når vi deklarerer en barneklasse. Eksempelvis kunne foreldreklassen til klassen 'Ost' vært kalt for Meieriprodukter, som videre hadde holdt på verdier som inkluderte alle meieriprodukter, og ikke bare egenskaper som assosieres med Ost klassen. Dette er en svært effektiv måte å skrive javakode på, da man kan bruke foreldreklasser til å deklareere felles attributter og egenskaper for alle meieriprodukter, slik at man ikke trenger å gjøre dette i hver enkel klasse som assosieres med meieriprodukter.

- Polymorphism

Ordet Polymorphism er hentet ut fra gresk og blir løst oversatt «Mange former».

Med dette så menes det at vi kan på flere forskjellige måter få en metode med samme navn til å funksjonere unikt i forhold til forskjellige klasser. To av disse teknikkene for å gjøre dette er «Override» og «Overload» som er forklart tidligere i dette dokumentet.

En siste måte jeg kjenner til å gjøre dette på er ved å bruke det som kalles «Interface Polymorphism». Interface er en, eller flere abstrakte metoder(metoder uten kropp) som blir definert med en unik egenskap i sin inneværende klasse. Når vi bruker «implements» i flere klasser kan vi knytte sammen disse metodene til å gi ut resultater i Main filen som knytter objektet til sin representative klasse.

Polymorphism er svært nyttig da dette kan brukes til å gjøre koden mer fleksibel og tilgjengelig når man jobber med objekter fra forskjellige klasser. Det er også mer effektivt, da vi åpner for muligheten til å bruke samme metode i forskjellige klasser med forskjellige utfall ut i fra metodens definisjon i den inneværende klassen. Dette er også med på å forenkle koden, slik at det er lettere for en utenforstående og lese/sette seg inn i kodestrukturen.

- private,public,(protected) (klasse,variabel,metode)

Dette er aksesserings moderatorer. De har som funksjon å definere hvor den gitte klassen, objektet, eller metoden skal kunne aksesseres.

- private deklarerer at det gitte elementet kun kan aksesseres innenfor klassen som holder på elementet.
- public deklarerer at det gitte elementet er tilgjengelig via alle klasser.
- protected deklarerer at det gitte elementet kun er tilgjengelig for klasser som ligger i samme pakkestruktur som inneholder klassen elementet tilhører.

- this og super

Nøkkelordet 'this' blir brukt for å referere til den gjeldende instansen i klassen. Det kan brukes i svært mange forskjellige sammenhenger, men hensikten med den er at det skal gjøres mulig for programvaren å kunne skille mellom instans variabler og en metodes parametere.

Nøkkelordet 'super' brukes for å referere til superklasser eller foreldreklasser(arv). Vi bruker super for å kunne kalle på metoder og egenskaper som er blitt konstruert i overordnede klasser. Vi kan via denne metoden blant annet opprette det som kalles for superkonstruktører. I eksempelet som klassen Ost ville vi kunne brukt 'super' for å hente ut verdier fra foreldreklassen Meieriprodukter når vi lager en konstruktør for klassen Ost.

Litteraturliste (Teorioppgaver)

Schildt, H. (2022). Java: A Beginner's Guide (9. Utg). McGraw Hill. ISBN: 978-1260463552

OpenAI. (2024). ChatGPT(ChatGPT 3.5)

<https://chat.openai.com/share/b93f6007-9a10-467e-b52b-066683a62c09>

Litteraturliste (Programmeringsoppgaver)

OpenAI. (2024). ChatGPT(ChatGPT 3.5)

<https://chat.openai.com/share/5910eb96-5ddd-43e1-851a-b6b70369038b>